

# Extending the Actor Model with Synchronous Delivery: **fast\_send** from CPU Threads to FPGA Pipelines

Vincent Maciejewski  
M2 Tech, Montreal  
mayeski@gmail.com

October 2026

## Abstract

In the classic actor semantics communication is asynchronous. Actor and active-object systems add synchronous communication as a request followed by a blocking wait, as a call confined to one scheduling unit, or as a rendezvous. We study a different primitive, **fast\_send**, from the open-source actor framework Kaspar-HFT: the receiver’s handler runs as a turn now, on the caller’s execution resource when possible and otherwise on a resource dedicated to the receiver, under the receiver’s mutual exclusion and without re-entry, and its reply is returned as a value; the handler is the same code for both deliveries. We extend the active-object calculus of De Koster and De Meuter with explicit execution resources and state the properties of **fast\_send** against their isolated turn principle. Isolation is kept. Determinism becomes relative to the values that **fast\_send** calls return. For terminating handlers under fair scheduling, a turn can fail to complete exactly when a wait-for cycle is reachable from its thread; without **fast\_send** there are none, which recovers the original liveness result. When execution resources are shared, **fast\_send** completes where a send followed by a blocking wait deadlocks, so the classic encodings of synchronous into asynchronous communication do not carry over. For debugging and deterministic replay, we prove that a single global queue driving every actor, in any language or on any substrate, is trace-sound: its traces are traces of the concurrent system, although deadlocks that need two turns in progress at once cannot appear in it. Finally we map **send** and **fast\_send** onto CPU threads, a Rust runtime and an FPGA actor runtime, and propose a mapping onto GPUs.

## 1 Introduction

An actor owns private state and communicates only by messages. In the classic formulations [7, 1, 2] there is exactly one way to communicate: an asynchronous send, which places a message in the receiver’s mailbox and returns at once. The receiver processes its messages one at a time, each in a *turn*. Because a turn never waits on anything, turns cannot deadlock, and because no state is shared, turns cannot race. De Koster and De Meuter name this the *isolated turn principle*, give an operational semantics for four families of actor languages, and prove the principle for each [6].

Real actor systems nonetheless need request/reply or synchronisation, and they obtain it in one of three ways. The sender sends a request and then blocks until a reply message arrives — the receiver still runs on its own thread (ABCL/1’s *now*-type messages [14], ABS calls between cogs [9], Erlang’s `gen_server:call`, Akka’s `ask`); or a synchronous call is allowed only between objects that already share one scheduling unit (ABS calls inside a cog, E’s immediate calls inside a vat [11]); or sender and receiver meet at a rendezvous (Extended Rebeca [13]).

**Kaspar-HFT and `fast_send`.** Kaspar-HFT<sup>1</sup> [10] is an open-source C++20 actor framework, written for high-frequency trading and used to build a trading system for CME futures: market data decoding, order-book reconstruction, execution algorithms and order routing are all actors. In Kaspar-HFT an actor is a C++ object that registers one handler per message type and holds references to other actors. It offers two ways to deliver a message. `send` is the classic asynchronous send: the message is placed in the receiver’s mailbox, the sender continues, and a thread that serves the receiver later runs its handler. `fast_send` is synchronous: the *sending* thread acquires the receiver’s lock, runs the receiver’s handler inline, bypassing its mailbox, and receives the handler’s reply as the call’s return value. The handler is the same code in both cases and does not depend on which delivery was used — the property the Kaspar-HFT paper calls *receiver transparency*. Kaspar-HFT adds two further extensions that matter here: actor *groups*, which run a set of actors on one thread behind one shared mailbox, and a choice of mailbox implementation per actor. That paper describes the framework and measures its cost; it does not give `fast_send` a formal semantics.

This paper studies `fast_send` as a third form of synchronous communication: the receiver’s turn is run *now*, by an execution resource that the caller is willing to wait on — on a CPU, the caller’s own thread — under the receiver’s exclusive access, and its result is returned to the caller as a value. It is neither a blocking wait on another thread’s work nor a call confined to one scheduling unit, nor a rendezvous. Synchronous communication in actor systems is not new; what this paper contributes is a precise semantics for this particular primitive, in which execution resources are explicit, and an exact account of what it does to isolation, determinism, liveness, message order and single-queue simulation when those resources are finite (Sections 4 to 6). We also show that it has a distinct mechanism on each runtime considered, just as asynchronous send does (Section 7).

In short, `fast_send` is the synchronous form that runs the receiver’s ordinary handler, reaches any actor across scheduling units, returns a value, and completes on a shared execution resource where send-then-wait deadlocks.

## Contributions.

1. A thread-aware extension of the active-object calculus of [6] with a `fast_send` rule, in two variants: the callee’s turn runs on the caller’s resource (FAST-LOCAL) or on the callee’s own resource while the caller waits (FAST-REMOTE) (Section 4). Both variants start the same method body as asynchronous delivery, which is receiver transparency stated in the calculus (Remark 3).
2. Its properties, stated against the isolated turn principle (Section 5): isolation is kept (Theorem 6); determinism holds relative to replies (Theorem 7); liveness holds exactly when no reachable wait-for cycle is reachable from the turn’s thread, under terminating handlers and fair scheduling (Theorem 10), and the original liveness result is the special case without `fast_send` (Corollary 11); a per-thread call-chain check detects exactly the cycles confined to one thread (Proposition 12); a `fast_send` is never queued behind the callee’s mailbox (Proposition 13); and `fast_send` is not send-then-wait when resources are shared (Proposition 14).
3. Trace soundness of a single global queue as a simulator for debugging and deterministic replay, across languages and substrates (Theorem 16), its limits (Proposition 18), and an offline check for the deadlocks it cannot exhibit (Proposition 19) (Section 6).
4. A mapping of both primitives onto CPU threads, a Rust runtime and an FPGA actor runtime, a proposed mapping onto GPUs, and the mappings between them (Section 7), with their implementation status in Kaspar-HFT (Section 8).

---

<sup>1</sup>Kaspar-HFT is open source: <https://github.com/vincent212/kaspar-hft>.

## 2 Background: turns and the isolated turn principle

De Koster and De Meuter classify actor languages into four families — classic actors, active objects, processes and communicating event loops — and give each a small-step operational semantics [6]. We build on their active-object calculus, because it matches the programming model of the runtimes studied here: an actor has a fixed interface, a FIFO mailbox and imperative private state.

In that calculus a configuration is a set of actors  $\mathcal{A}(\iota_a, \mu, e, O, o)$ : an identifier, a mailbox  $\mu$  of messages  $\mathcal{M}(msg, \bar{v})$ , the expression  $e$  the actor is evaluating, a heap  $O$  of passive objects it owns, and an active object  $o$  that fixes its interface. Actor-local rules (LET, NEW, INVOKE, FIELD-ACCESS, FIELD-UPDATE, SELF-SEND, RECEIVE) reduce one actor; actor-global rules (SEND, SPAWN) involve two. SEND appends a message to the receiver’s mailbox after deep-copying the passive objects it references (the function `pass`), so no state is shared. RECEIVE starts a turn: when an actor’s expression is a value, the head of its mailbox is removed and the matching method body becomes its expression. A *turn* is the sequence of reductions of one actor between two applications of RECEIVE.

The isolated turn principle then consists of two theorems [6]. *Safety*: the value a turn reduces to is determined up to  $\alpha$ -equivalence, whatever the interleaving with other actors, and a turn is not interleaved with another turn of the same actor. *Liveness*: a turn never blocks — every reducible expression of a turn can take a step — so turns are free of deadlock. The calculi contain no blocking operation; this is what makes liveness hold.

Two features of this setting matter below. The calculus has no notion of the thread or processor that performs a reduction: each actor reduces independently, as if it had its own. And every communication is a SEND. A synchronous call needs the first to be made explicit and adds a second kind of communication.

## 3 Synchronous communication in actor systems

Three reasons are usually given for leaving synchronous calls out of the actor model. *Order*: in Hewitt’s model messages are guaranteed to arrive but in no particular order, so “handle this now, before what I do next” has no meaning. *Deadlock*: an actor that handles one message at a time and waits inside a turn can wait on itself, directly or through others. *Distribution*: across machines, a synchronous call can wait forever on a failed peer.

Where systems provide synchronous communication anyway, they do so in one of three ways.

**Wait on the receiver’s thread.** ABCL/1 distinguishes *past* (asynchronous), *now* (send and wait for the reply) and *future* messages [14]. In ABS [9], a synchronous call  $v = o.m(e)$  between objects in different cogs abbreviates  $x = o!m(e)$ ;  $v = x.get$ : an asynchronous call followed by a blocking `get`, which “blocks all execution in the object until the return value is available”. Erlang’s `gen_server:call` and Akka’s `ask` are library forms of the same pattern, usually with a timeout. In all of these the receiver’s turn runs on the receiver’s own thread; the caller’s thread waits.

**Call within one scheduling unit.** In ABS, a synchronous call between objects in the same cog has “the reentrant semantics known from, e.g., Java threads” [9]: control passes to the callee on the cog’s single processor. In E, immediate calls are allowed only on near references, to objects in the same vat; between vats only eventual sends exist [11]. These calls run on the caller’s processor, but they never cross a scheduling unit, so they need no mutual exclusion; and ABS allows re-entry into an object that is already executing.

Table 1: Synchronous communication in actor and active-object systems. “Busy callee”: what happens when the callee is in a turn when the call is made.

| Mechanism                                                                            | Who runs the callee’s code                            | Reach          | Reply value | Re-entry     | Busy callee                                        |
|--------------------------------------------------------------------------------------|-------------------------------------------------------|----------------|-------------|--------------|----------------------------------------------------|
| Send-then-wait (ABCL/1 <i>now</i> , ABS between cogs, <code>gen_server:call</code> ) | the callee’s own processor, in a later turn           | any actor      | yes         | no           | request queued behind its mailbox                  |
| Call in one unit (ABS inside a cog, E immediate call)                                | the caller’s processor                                | same cog / vat | yes         | yes          | not applicable: one processor                      |
| Rendezvous (Extended Rebeca)                                                         | both, at a <b>receive</b> statement in the callee     | same component | no          | no self-send | sender waits for the <b>receive</b>                |
| <code>fast_send</code> (this paper)                                                  | the caller’s resource, or one dedicated to the callee | any actor      | yes         | no           | caller waits for the turn to end; mailbox bypassed |

**Rendezvous inside a component.** Extended Rebeca [13] groups actors (“rebecs”) into components. Components communicate only asynchronously; inside a component, rebecs may also exchange synchronous messages. A synchronous message has a signature but no handler: it is a “handshake between the execution of a send-statement by the caller and a receive statement by the callee”, and both sides block until they meet at that point; the data flows from sender to receiver only, and a rebec may not send a synchronous message to itself. The receiver takes part explicitly, by executing a **receive** statement inside one of its methods.

**Expressiveness.** Honda and Tokoro [8] and Boudol [4] showed that synchronous communication can be encoded in the asynchronous  $\pi$ -calculus. `fast_send` therefore adds nothing to what can be computed. The encodings assume, however, that every process makes progress independently — that execution resources are unbounded. Proposition 14 shows that once resources are shared, as in an actor group or a cog, send-then-wait deadlocks where `fast_send` completes. The difference between the primitives is a difference in who executes a turn and when.

**Where `fast_send` sits.** `fast_send` runs the receiver’s turn on a resource the caller is willing to wait on — on a CPU, the caller’s own thread — across scheduling units, under the receiver’s mutual exclusion, and without re-entry: an actor already in a turn cannot be entered again, and a call that tries waits, possibly forever. It combines the “run it here” of calls within a scheduling unit with the “any actor” of send-then-wait, and pays for it with a deadlock case that calls within a scheduling unit do not have. Unlike a rendezvous, the receiver does not take part: its ordinary handler runs as a whole turn and its result comes back as a value. Table 1 summarises the differences; the rest of the paper makes them precise.

## 4 A thread-aware calculus with `fast_send`

We extend the active-object calculus of [6] in three ways: execution resources become explicit, an actor’s current expression moves from the actor into a *frame* on a resource’s stack, and a `fast_send`

expression is added. We call the resources *threads*; on an FPGA they are hardware processes (Section 7).

#### 4.1 Semantic entities and syntax

$$\begin{aligned}
K \in \text{Configuration} &::= \langle A, T \rangle \\
a \in A \subseteq \text{Actor} &::= \mathcal{A}\langle \iota_a, \mu, O, o, \theta \rangle \\
t \in T \subseteq \text{Thread} &::= \mathcal{T}\langle \theta, S \rangle \quad S ::= \epsilon \mid S \cdot f \\
f \in \text{Frame} &::= \mathcal{F}\langle \iota_a, e, \kappa \rangle \quad \kappa ::= \text{turn} \mid \text{ret}(\rho)
\end{aligned}$$

An actor has an identifier, a FIFO mailbox  $\mu$ , a heap  $O$ , an active object  $o$ , and a *home thread*  $\theta$ . Several actors may share a home thread; that is an actor group. A thread has an identifier and a stack  $S$  of frames, top on the right. A frame records which actor it belongs to, the expression being evaluated, and why it exists: **turn** for a turn started from the mailbox, or **ret**( $\rho$ ) for a turn started by the **fast\_send** call  $\rho$ , whose value must be returned. Call identifiers  $\rho$  are fresh.

The expression syntax of the active-object calculus is extended with one form, and the run-time syntax with one more:

$$e ::= \dots \mid \text{fastsend}(e, \text{msg}, \bar{e}) \quad e ::= \dots \mid \text{wait}(\rho) \quad (\text{run time})$$

Evaluation contexts  $\mathcal{E}$  extend the originals left to right through the arguments of **fastsend**. No rule reduces **wait**( $\rho$ ): a frame whose expression has the form  $\mathcal{E}[\text{wait}(\rho)]$  is *waiting*.

**Definition 1** (Busy). Actor  $\iota_a$  is *busy* in  $K$ , written  $\text{busy}_K(\iota_a)$ , if some thread of  $K$  has a frame of  $\iota_a$  on its stack.

For an actor with active object  $o = \mathcal{O}\langle \iota_o, \text{cls}, \bar{v}_f \rangle$  and a message  $\mathcal{M}\langle \text{msg}, \bar{v} \rangle$ , let **body**( $a, \text{msg}, \bar{v}$ ) be the body of method  $\text{msg}$  of  $\text{cls}$  with fields, parameters and **self** substituted, exactly as in **RECEIVE** and **INVOKE** of [6].

#### 4.2 Reduction rules

The actor-local rules of [6] apply unchanged to the *top* frame of any thread, reading and writing the heap of the frame's actor. **SEND** and **SELF-SEND** likewise apply to a top frame and append to the receiver's mailbox. **SPAWN** creates an actor with an empty mailbox and a home thread, either an existing one or a fresh  $\mathcal{T}\langle \theta', \epsilon \rangle$ . **RECEIVE** and the end of a turn become thread rules, and three rules are new (Figure 1).

- **RECEIVE**: an idle thread starts a turn of one of its idle actors whose mailbox is not empty. With several actors on one thread, the choice among them is free.
- **TURN-END**: a turn started from the mailbox that has reduced to a value is removed. Its value is discarded; a reply to a **send** is an explicit **send**.
- **FAST-LOCAL**: the caller's frame becomes waiting, and the callee's turn is pushed on the *caller's* stack. This is **fast\_send** on one CPU thread, or inside one FPGA process.
- **FAST-REMOTE**: the caller's frame becomes waiting, and the callee's turn starts on the callee's own, idle, home thread. This is **fast\_send** to an actor on another resource that the caller waits on: a CPU thread calling into an FPGA process, an FPGA process calling a CPU actor, or a call to another machine.

$$\begin{array}{c}
\frac{t = \mathcal{T}\langle\theta, \epsilon\rangle \quad a = \mathcal{A}\langle\iota_a, \mathcal{M}\langle msg, \bar{v} \rangle \cdot \mu, O, o, \theta \rangle \quad \neg \text{busy}_K(\iota_a)}{K \longrightarrow K[t \mapsto \mathcal{T}\langle\theta, \mathcal{F}\langle\iota_a, \text{body}(a, msg, \bar{v}), \text{turn}\rangle\rangle][a \mapsto \mathcal{A}\langle\iota_a, \mu, O, o, \theta\rangle]} \text{RECEIVE} \\
\\
\frac{t = \mathcal{T}\langle\theta, S \cdot \mathcal{F}\langle\iota_a, v, \text{turn}\rangle\rangle}{K \longrightarrow K[t \mapsto \mathcal{T}\langle\theta, S\rangle]} \text{TURN-END} \qquad \frac{\text{Call}(t, \iota_b, \rho)}{K \longrightarrow K[t \mapsto \mathcal{T}\langle\theta, S \cdot W_\rho \cdot C_\rho\rangle]} \text{FAST-LOCAL} \\
\\
\frac{\text{Call}(t, \iota_b, \rho) \quad \theta_b \neq \theta \quad t_b = \mathcal{T}\langle\theta_b, \epsilon\rangle}{K \longrightarrow K[t \mapsto \mathcal{T}\langle\theta, S \cdot W_\rho\rangle][t_b \mapsto \mathcal{T}\langle\theta_b, C_\rho\rangle]} \text{FAST-REMOTE} \\
\\
\frac{t' = \mathcal{T}\langle\theta', S' \cdot \mathcal{F}\langle\iota_b, v, \text{ret}(\rho)\rangle\rangle \quad t = \mathcal{T}\langle\theta, S \cdot \mathcal{F}\langle\iota_a, \mathcal{E}[\text{wait}(\rho)], \kappa\rangle \cdot S''\rangle \quad v' = \text{pass}(v)}{K \longrightarrow K[t' \mapsto \mathcal{T}\langle\theta', S'\rangle][t \mapsto \mathcal{T}\langle\theta, S \cdot \mathcal{F}\langle\iota_a, \mathcal{E}[v'], \kappa\rangle \cdot S''\rangle]} \text{FAST-RETURN}
\end{array}$$

Figure 1: Thread rules.  $K[t \mapsto t']$  replaces thread  $t$  by  $t'$  (and likewise for actors). The two **fast\_send** rules share the premise  $\text{Call}(t, \iota_b, \rho)$ , which holds when  $t = \mathcal{T}\langle\theta, S \cdot \mathcal{F}\langle\iota_a, \mathcal{E}[\text{fastsend}(\iota_b, msg, \bar{v})], \kappa\rangle\rangle$ ,  $\neg \text{busy}_K(\iota_b)$  and  $\rho$  is fresh; with  $b = \mathcal{A}\langle\iota_b, \mu_b, O_b, o_b, \theta_b\rangle$  and  $\bar{v}' = \text{pass}(\bar{v})$ , the caller's waiting frame is  $W_\rho = \mathcal{F}\langle\iota_a, \mathcal{E}[\text{wait}(\rho)], \kappa\rangle$  and the callee's frame is  $C_\rho = \mathcal{F}\langle\iota_b, \text{body}(b, msg, \bar{v}'), \text{ret}(\rho)\rangle$ . In FAST-RETURN,  $t$  and  $t'$  may be the same thread, in which case  $S''$  is empty and the returned frame was directly above the waiting one.

- FAST-RETURN: a callee turn that has reduced to a value is removed and its value, copied, replaces the  $\text{wait}(\rho)$  of its caller.

Two different conditions govern a **fast\_send**, and the distinction matters throughout. *Actor availability*: both rules require the callee actor to be idle, that is, to have no frame on any thread. *Resource availability*: FAST-REMOTE additionally requires the callee's home thread to be idle. An actor can be idle while its home thread is busy running another actor's turn; FAST-LOCAL then still applies, because it does not use the callee's thread, but FAST-REMOTE does not. When the condition of the chosen rule fails, the calling frame is *blocked*. The callee's mailbox is never consulted. A system may use FAST-LOCAL for some pairs of actors and FAST-REMOTE for others; every result below holds for any such mixture. Figure 2 shows **send** and FAST-LOCAL side by side.

*Remark 2* (What **fast\_send** does not provide). **fast\_send** is not a function call. It does not enter an actor that is already in a turn: re-entry is excluded, and a call to a busy actor waits. It gives no guarantee that the call completes (Theorem 10), and hence no deadlock freedom. It is not a priority mechanism: it is not queued behind the callee's mailbox (Proposition 13) but does not interrupt a turn in progress. It carries no timing guarantee, no failure handling — a call to an actor whose resource has failed waits forever — and no transactional semantics across the actors a turn calls.

*Remark 3* (Receiver transparency). A method body cannot tell how its turn started: RECEIVE, FAST-LOCAL and FAST-REMOTE all install the same  $\text{body}(b, msg, \bar{v})$ . This is the calculus counterpart of receiver transparency in Kaspar-HFT [10]. It is a property of the interface a turn is given, not a claim about what code in a real language can observe about its execution context.

## 5 Properties

Throughout,  $K_0 \longrightarrow^* K$  ranges over configurations reachable from an initial configuration with one actor, as in [6], whose single thread holds no frames.

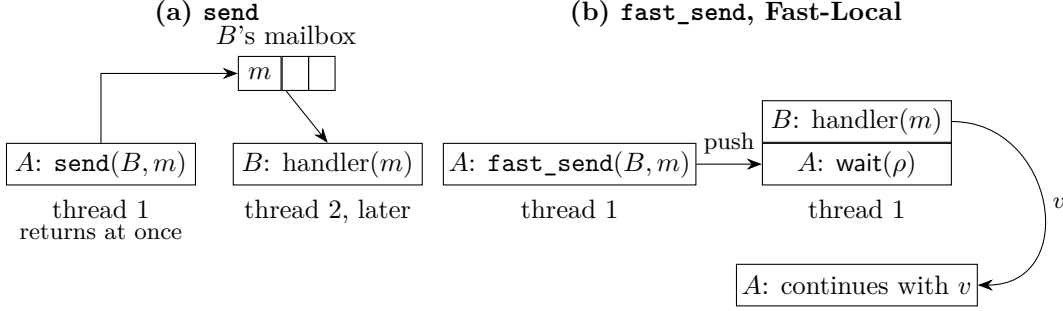


Figure 2: (a) **send** enqueues the message and returns; the receiver’s handler runs later on a thread that serves it. (b) **fast\_send** under FAST-LOCAL: the caller’s frame waits, the callee’s turn is pushed on the caller’s own thread, and its value  $v$  is returned when it ends. The callee’s mailbox is not used.

## 5.1 Isolation

**Lemma 4** (One frame per actor). *In every reachable configuration, each actor has at most one frame, and every frame reads and writes only the heap of its own actor.*

*Proof.* Initially there are no frames. Only RECEIVE, FAST-LOCAL and FAST-REMOTE create frames, and each creates a frame for an actor that is not busy, that is, has no frame. TURN-END and FAST-RETURN only remove frames; the other rules rewrite the expression of an existing top frame. The heap discipline is that of the active-object calculus: field access and update apply to the heap of the frame’s actor, and values crossing actors (SEND, the two **fast\_send** rules, FAST-RETURN) are copied by **pass**.  $\square$

**Definition 5** (Turn). A turn of  $\iota_a$  is the sequence of reductions from the creation of a frame of  $\iota_a$  (by RECEIVE, FAST-LOCAL or FAST-REMOTE) to its removal (by TURN-END or FAST-RETURN).

**Theorem 6** (Isolation). *No two turns of the same actor overlap, and during a turn of  $\iota_a$  the heap of  $\iota_a$  is changed only by reductions of that turn’s frame.*

*Proof.* Immediate from Lemma 4: a second turn would need a second frame.  $\square$

This is the safety half of the isolated turn principle, and it holds with **fast\_send**. A turn that makes a **fast\_send** is suspended while the callee runs, but no other turn of the same actor can run in the meantime, and the callee cannot touch the caller’s state.

## 5.2 Determinism relative to replies

In [6] a turn’s value is determined, because a turn receives no input after it starts. With **fast\_send** it does: each FAST-RETURN substitutes a value that depends on the callee’s state, which other turns may have changed before the call.

**Theorem 7** (Determinism relative to replies). *Consider two turns of the same actor that start from  $\alpha$ -equivalent heaps and expressions. If the values substituted into them by FAST-RETURN, in order, are  $\alpha$ -equivalent, then the two turns end with  $\alpha$ -equivalent values and heaps and issue  $\alpha$ -equivalent **send** and **fast\_send** requests in the same order.*

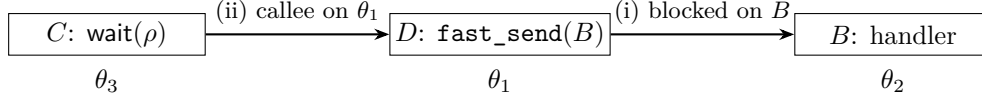


Figure 3: Wait-for edges in one configuration.  $C$  on  $\theta_3$  made a **fast\_send** to  $D$  under FAST-REMOTE, so  $D$ 's turn (a frame marked  $\text{ret}(\rho)$ ) runs on  $\theta_1$  and  $\theta_3$  waits for it: an edge of kind (ii).  $D$ 's top frame is a **fast\_send** to  $B$ , which is busy on  $\theta_2$ : an edge of kind (i).  $\theta_2$  has no outgoing edge, so its top frame can be reduced; when  $B$ 's turn ends,  $D$ 's call can start, and the path shortens from the right. With no cycle on the path, every thread on it eventually progresses (Theorem 10).

*Proof.* As the safety proof of [6], by induction on the reductions of the turn's frame: evaluation contexts fix the order of reduction, and every actor-local rule produces  $\alpha$ -equivalent results from  $\alpha$ -equivalent inputs. The new cases are the two **fast\_send** rules, which rewrite the frame to  $\mathcal{E}[\text{wait}(\rho)]$  and perform no reduction on it until FAST-RETURN, whose substituted value is  $\alpha$ -equivalent by hypothesis. Reductions of other frames do not touch this turn's heap (Lemma 4).  $\square$

A turn that makes no **fast\_send** satisfies the original safety theorem unchanged. A turn that does is deterministic as a function of its replies, as it would be if the replies were messages; what **fast\_send** removes is the possibility of any other message of the same actor being handled between the request and the reply.

### 5.3 Liveness

**Definition 8** (Blocked frame, wait-for graph). A top frame  $\mathcal{F}(\iota_a, \mathcal{E}[\text{fastsend}(\iota_b, \text{msg}, \bar{v})], \kappa)$  on thread  $\theta$  is *blocked* in  $K$  if the **fast\_send** rule used for that call does not apply to it. The *wait-for graph*  $W_K$  has the threads of  $K$  as nodes and an edge  $\theta \rightarrow \theta'$  when either

- (i) the top frame of  $\theta$  is blocked on  $\iota_b$  and  $\theta'$  holds the frame of  $\iota_b$ ; or it is blocked by FAST-REMOTE because  $\iota_b$ 's home thread  $\theta'$  is not idle; or
- (ii)  $\theta$  holds a waiting frame  $\mathcal{E}[\text{wait}(\rho)]$  whose callee frame  $\text{ret}(\rho)$  is on  $\theta' \neq \theta$ .

Figure 3 shows both kinds of edge. Every edge leaves a thread that cannot progress until something on the target thread is removed. A thread whose top frame is waiting has an edge of kind (ii) unless the callee is above it on the same stack, in which case the thread is not stuck: its top frame is the callee's.

The liveness result is not a termination theorem for arbitrary actor programs. We assume that every handler terminates once the **fast\_send** calls it makes have returned, and that scheduling is fair. Fairness has two parts. The first is the usual one: a thread whose top frame can be reduced is eventually reduced. The second concerns a blocked call: when a busy callee's turn ends, the calculus lets its home thread start its next mailbox turn before the waiting **fast\_send** applies, and this can repeat for ever, starving the caller. We therefore also assume that a **fast\_send** that becomes applicable infinitely often is eventually applied. A runtime whose lock gives no fairness guarantee does not provide this; the preference for pending calls in Proposition 13 does. Under these assumptions the only source of non-termination that remains is cyclic waiting, which **fast\_send** introduces; a handler that loops forever violates the assumption and is outside the result.

**Assumption 9** (Terminating handlers, fair scheduling). (a) Every method body terminates when its **fast\_send** calls return. (b) Every thread whose top frame can be reduced is eventually reduced. (c) A blocked **fast\_send** whose rule becomes applicable infinitely often is eventually applied.

**Theorem 10** (Liveness and wait-for cycles). *Under Assumption 9, a turn in a reachable configuration  $K$  fails to complete in some fair continuation of  $K$  if and only if, in some configuration reachable from  $K$ , its thread has a path in the wait-for graph to a cycle.*

*Proof.* ( $\Leftarrow$ ) Take the cycle  $\theta_1 \rightarrow \dots \rightarrow \theta_n \rightarrow \theta_1$ . Each  $\theta_i$  waits for a frame on  $\theta_{i+1}$  to be removed. That frame is below the top of  $\theta_{i+1}$  or is its top; in either case its removal needs  $\theta_{i+1}$  to progress, which needs a frame on  $\theta_{i+2}$  removed, and so on around the cycle. No rule removes a frame of a thread whose top frame is blocked or waiting, so none of the  $\theta_i$  ever progresses, and a thread with a path to the cycle waits on it forever.

( $\Rightarrow$ ) Under Assumption 9, a frame that can be reduced is eventually reduced and, its body terminating, eventually removed. So a turn that never completes must, from some point on, sit on a thread that is permanently blocked or waiting, and we show that this requires a path to a cycle. Suppose not: in every configuration reachable from  $K$ , the path from  $\theta$  in  $W$  is finite and acyclic. Its last thread  $\theta^*$  has no outgoing edge, so its top frame is neither blocked nor waiting on another thread, and it is reduced until it is removed by TURN-END or FAST-RETURN. Each such removal either brings  $\theta^*$  closer to removing the frame its predecessor on the path waits for — there are finitely many frames above it — or removes that frame, after which the predecessor’s waiting frame resumes or its blocked **fast\_send** becomes applicable, and by part (c) of Assumption 9 is eventually applied. Induction on the length of the path then shows that  $\theta$  itself progresses, contradicting the assumption that its turn never completes.  $\square$

**Corollary 11** (Liveness without **fast\_send**). *If a program never evaluates **fastsend**, every turn completes under Assumption 9.*

*Proof.* Without **fast\_send** there are no blocked or waiting frames, so the wait-for graph has no edges.  $\square$

This is the liveness half of the isolated turn principle, recovered as the case without **fast\_send**. With **fast\_send**, liveness is a property of the program’s pattern of synchronous calls rather than of the model.

**Proposition 12** (What a per-thread call chain detects). *Suppose a thread checks, before each **fast\_send** to  $\iota_b$ , whether  $\iota_b$  has a frame on its own stack. The check succeeds exactly for the self-loops  $\theta \rightarrow \theta$  of the wait-for graph. It does not detect cycles of length two or more.*

*Proof.* A self-loop arises when the top frame of  $\theta$  is blocked on an actor whose frame is on  $\theta$  itself, which is what the check tests. A cycle through two threads  $\theta_1 \rightarrow \theta_2 \rightarrow \theta_1$  arises when the top frame of each is blocked on an actor whose frame is on the other: each check inspects only its own stack, finds the callee absent, and passes. For example, thread 1 runs a **fast\_send** to  $A$  whose body makes a **fast\_send** to  $B$ , while thread 2 runs a **fast\_send** to  $B$  whose body makes a **fast\_send** to  $A$ .  $\square$

Figure 4(a) shows the two-thread case. A per-thread detector, such as the call-chain test designed in [10],<sup>2</sup> therefore turns same-thread cycles into errors and leaves cross-thread cycles as deadlocks. Detecting those needs shared state: a global wait-for graph, a global order on actors, or bounded waiting with a fallback to asynchronous delivery.

<sup>2</sup>Kaspar-HFT does not implement the call-chain test yet: **fast\_send** takes the receiver’s mutex directly, so a synchronous cycle currently deadlocks.

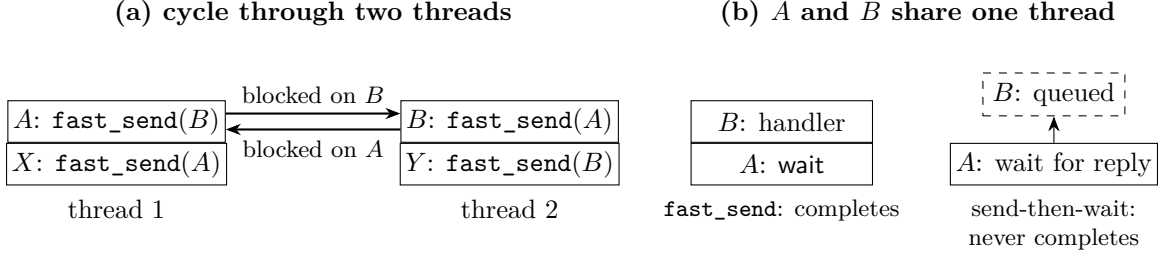


Figure 4: (a) A wait-for cycle through two threads: each top frame is blocked on an actor whose frame is on the other thread. A check of each thread’s own stack passes on both (Proposition 12). (b) When  $A$  and  $B$  share a thread, `fast_send` pushes  $B$ ’s turn on it and completes; send-then-wait leaves the request in  $B$ ’s mailbox behind  $A$ ’s waiting turn, and  $B$  never runs (Proposition 14).

## 5.4 Order

**Proposition 13** (Order). (1) Messages sent with `send` by the same actor  $\iota_a$  to the same receiver  $\iota_b$  start turns of  $\iota_b$  in the order  $\iota_a$  sent them. Nothing is claimed about the relative order of messages from different senders. (2) A `fast_send` from  $\iota_a$  to  $\iota_b$  is never placed in  $\iota_b$ ’s mailbox: its turn starts when a `fast_send` rule applies, whatever the mailbox holds, so it can start before messages already queued for  $\iota_b$ , including earlier `sends` from  $\iota_a$ . While  $\iota_b$  is busy, the calculus leaves open whether the pending call or the head of the mailbox starts next; a runtime that always prefers a pending `fast_send` guarantees that the call starts before every message that was in  $\iota_b$ ’s mailbox when it was made. (3) Messages sent during the callee’s turn of a `fast_send` are enqueued before any message the caller sends after the `fast_send` returns.

*Proof.* (1) The sends of  $\iota_a$  are totally ordered, because  $\iota_a$  has at most one frame (Lemma 4); SEND appends and RECEIVE removes the head. (2) The `fast_send` rules start the callee’s turn without consulting its mailbox, and RECEIVE does not consult pending calls; when both are enabled for  $\iota_b$  the choice is free. Under the preference, each time  $\iota_b$  becomes idle while the call is pending, a `fast_send` rule is applied to  $\iota_b$  before any RECEIVE of  $\iota_b$ , so no queued message of  $\iota_b$  starts first. (3) The caller’s frame is waiting from the `fast_send` until FAST-RETURN, and the callee’s sends happen before that.  $\square$

Part (2) is the formal counterpart of “`fast_send` bypasses the mailbox”, and it is one of the practical reasons to use `fast_send`: a call is not delayed by the backlog in the callee’s mailbox. It is not a priority mechanism — a turn in progress is never interrupted (Remark 2). The FPGA runtime of Section 7 implements the preference: each actor process serves a dedicated `fast_send` input port before its mailbox FIFO.

## 5.5 `fast_send` is not send-then-wait

Let *send-then-wait* be the pattern of the first group in Section 3: the caller sends a request with `send` and its frame then blocks until a reply message for it arrives. We model the blocking by a frame that no rule reduces until the reply has been enqueued in the caller’s mailbox; the frame stays on its thread.

**Proposition 14** (Shared resources). Let actors  $\iota_a$  and  $\iota_b$  share a home thread, and let  $\iota_b$ ’s method return without further communication. A turn of  $\iota_a$  that obtains  $\iota_b$ ’s result by *send-then-wait* never completes. The same turn using `fast_send` with FAST-LOCAL completes.

*Proof.* With send-then-wait, the request is enqueued in  $\iota_b$ 's mailbox and  $\iota_a$ 's frame stays on the shared thread  $\theta$ . RECEIVE for  $\iota_b$  requires  $\theta$  to have an empty stack, which it never has again, so the reply is never produced. With **fast\_send**, FAST-LOCAL pushes  $\iota_b$ 's turn on  $\theta$  above  $\iota_a$ 's frame; it terminates and FAST-RETURN resumes  $\iota_a$ .  $\square$

Figure 4(b) illustrates the proposition. The encodings of synchronous into asynchronous communication [8, 4] are sound in a setting where every process progresses independently. In a setting with a bounded number of execution resources — actor groups, ABS cogs, E vats, a pinned thread per core, a fixed number of FPGA processes — **fast\_send** and send-then-wait differ in whether they complete. The proposition also shows why a synchronous call inside one scheduling unit, as in ABS cogs and E vats, has to run on the caller's processor: inside the unit, send-then-wait cannot complete.

## 6 Simulation and debugging from a global queue

Deterministic replay runs recorded external inputs through the same actor handlers used in production; in a trading system this is a backtest. A debugger needs the same thing plus the ability to stop, step and inspect. Both are easiest when every actor in the system — whatever language it is written in and whatever substrate it normally runs on — is driven from a single queue by a single thread: every message goes into the queue, receivers take their messages from it, and the execution is one deterministic sequence that can be replayed, paused and inspected. The Kaspar-HFT actor group, which co-schedules actors behind one mailbox, already serves as such a simulator for CPU actors [10].<sup>3</sup> The question is whether **fast\_send** breaks the correspondence between that simulator and the concurrent system.

### 6.1 The global-queue semantics

The global-queue semantics  $GQ$  is the calculus of Section 4 with one thread  $\theta_0$ , the home thread of every actor, and with per-actor mailboxes replaced by one FIFO queue  $Q$  of pairs  $(\iota_b, m)$ :

- SEND appends  $(\iota_b, \mathcal{M}\langle msg, \text{pass}(\bar{v}) \rangle)$  to  $Q$ ;
- RECEIVE-GQ: when  $\theta_0$  has an empty stack, remove the head  $(\iota_b, m)$  of  $Q$  and start  $\iota_b$ 's turn for  $m$ ;
- **fast\_send** uses FAST-LOCAL on  $\theta_0$ .

An actor that normally runs on another substrate takes part through its method bodies alone: an FPGA actor's turn is its step function, executed by the simulator in software; a Rust actor's turn is its handler, called across the language boundary. By receiver transparency, the body is the same whatever starts the turn.

The production semantics  $P$  is the calculus of Section 4 with any assignment of actors to threads and any choice of FAST-LOCAL or FAST-REMOTE per call site, subject to one condition on the resources that FAST-REMOTE uses:

**Assumption 15** (Dedicated remote resources). A thread that serves as the home thread of a FAST-REMOTE callee is the home of that actor only.

---

<sup>3</sup>The cross-language, cross-substrate global queue described in this section is not implemented in Kaspar-HFT yet; CPU actors in one group already run this way.

Which runtimes satisfy it:

- *CPU to FPGA, and between FPGA processes*: each FPGA actor is its own process, so the assumption holds. Between FPGA processes the host only relays the request and the reply; it does not run the callee.
- *Within the CPU runtime*: `fast_send` is FAST-LOCAL, so the assumption does not apply.
- *FPGA to CPU*: in the current bridge one host thread runs every CPU handler that FPGA actors call, so that resource is shared and the assumption fails. Such a call can wait for the bridge where the global queue proceeds — for instance when a CPU handler running on the bridge calls into the FPGA and the FPGA actor calls back to the CPU. One bridge thread per called CPU actor, or running the handler under FAST-LOCAL semantics, would restore it.
- *Another machine*: the assumption holds if the remote actor has a thread of its own, and fails if that thread also serves other actors, for example a group.

Where the assumption fails, Theorem 16 does not cover the calls concerned.

## 6.2 Soundness

Say that two executions *agree* if every actor starts the same turns in the same order, with  $\alpha$ -equivalent messages, and every turn ends with an  $\alpha$ -equivalent value and heap.

**Theorem 16** (Trace soundness of the global queue). *Under Assumption 15, for every finite execution of  $GQ$  from a given initial configuration there is an execution of  $P$  from the corresponding initial configuration that agrees with it: the traces of  $GQ$  are included in those of  $P$ . In particular, a configuration in which a turn is stuck, or an error state, reached in  $GQ$  has an agreeing counterpart in  $P$ .*

*Proof.* We build the  $P$  execution step by step, maintaining the invariant that at every point where  $\theta_0$  has an empty stack in  $GQ$ , every thread of  $P$  has an empty stack, the heaps agree, and each actor's mailbox in  $P$  is the projection of  $Q$  onto that actor, in order.

*Receive.*  $GQ$  takes the head  $(\iota_b, m)$  of  $Q$ . By the invariant,  $m$  is the head of  $\iota_b$ 's mailbox in  $P$ , since every earlier pair for  $\iota_b$  in  $Q$  has already been removed.  $\iota_b$ 's home thread is idle and  $\iota_b$  is not busy, so `RECEIVE` applies in  $P$  to the same message.

*Within a turn.*  $P$  reduces only the thread that holds the corresponding frame, mirroring each  $GQ$  step: actor-local steps are identical; `SEND` appends to the receiver's mailbox, which preserves the projection invariant.

*fast\_send.* In  $GQ$ , FAST-LOCAL to  $\iota_b$  applies if and only if  $\iota_b$  has no frame on  $\theta_0$ 's stack. In the mirrored  $P$  execution the only frames in existence are the ones mirroring that stack, so  $\iota_b$  is busy in  $P$  exactly when it is busy in  $GQ$ . If  $P$  uses FAST-LOCAL for the call, it applies. If it uses FAST-REMOTE, the rule also needs  $\iota_b$ 's home thread to be idle. By Assumption 15 a frame reaches that thread only as a turn of  $\iota_b$  or stacked above one;  $\iota_b$  has no frame, so the thread is idle. The callee's turn then proceeds as above, and FAST-RETURN applies in both.

If  $GQ$  is stuck on a `fast_send` to a busy actor, the mirrored  $P$  frame is blocked on the same actor. Agreement of values and heaps follows from Theorem 7, since the mirrored turns receive the same replies.  $\square$

**Corollary 17** (What the simulator shows is real). *Every value a `fast_send` returns in  $GQ$ , every assertion failure, error state or same-thread `fast_send` cycle it reaches, can occur in the concurrent system. A  $GQ$  run is deterministic given its inputs, so it can be replayed exactly.*

Table 2: The global queue as a simulator of the concurrent system (under Assumption 15).

| Question                                                     | Answer | Result                      |
|--------------------------------------------------------------|--------|-----------------------------|
| Can it produce an execution the concurrent system cannot?    | no     | Theorem 16                  |
| Can it reproduce every execution of the concurrent system?   | no     | Proposition 18              |
| Can it exhibit a deadlock needing two mailbox turns at once? | no     | Proposition 18              |
| Can it exhibit a same-thread <b>fast_send</b> cycle?         | yes    | Theorem 16                  |
| Is a run reproducible from its inputs?                       | yes    | single queue, single thread |

So **fast\_send** does not break the global-queue simulation. What makes the argument go through is that in a single-threaded run the callee of a **fast\_send** is idle unless it is already on the current call chain: the simulator only ever takes the branch of **fast\_send** in which the callee was free, which is one of the branches the concurrent system can take. It is not obvious in general: without Assumption 15, a FAST-REMOTE callee whose resource is occupied by another actor blocks in  $P$  where  $GQ$  proceeds, and the simulator would then show behaviour the concurrent system cannot produce.

### 6.3 What the simulator cannot show

**Proposition 18** (Incompleteness). *There are executions of  $P$  that agree with no execution of  $GQ$ . Two examples: (1) the deadlock of Figure 4(a), where the turns of  $X$  and  $Y$ , both started from mailboxes, never end; (2) a turn of  $\iota_a$  that sends  $m$  to  $\iota_b$  and then makes a **fast\_send** to  $\iota_b$ , where  $\iota_b$ 's home thread runs the turn for  $m$  before the **fast\_send** applies, so the call sees  $\iota_b$ 's state after  $m$ .*

*Proof.* In  $GQ$ , RECEIVE- $GQ$  needs an empty stack, so at most one turn started from a mailbox is unfinished at any time, and every other turn in progress is a **fast\_send** callee on the same stack. (1) has two unfinished turns started from mailboxes, so an agreeing  $GQ$  execution would need both, which it cannot have. In (2),  $m$  enters  $Q$  during  $\iota_a$ 's turn and can start only after that turn ends, so in every  $GQ$  execution  $\iota_b$  starts the turn for the **fast\_send** before the turn for  $m$ ; in the  $P$  execution it starts them in the other order.  $\square$

Table 2 summarises what the global queue does and does not give. Replay can therefore confirm bugs but cannot rule out the two classes above. The first can be approached offline:

**Proposition 19** (Cycles in the call graph). *Let  $G$  be the directed graph on actors with an edge  $\iota_a \rightarrow \iota_b$  whenever some turn of  $\iota_a$  can evaluate a **fast\_send** to  $\iota_b$ . If  $G$  is acyclic, no execution of  $P$  reaches a wait-for cycle.*

*Proof.* The frames on any thread, bottom to top, belong to a path in  $G$ , since each frame above the bottom one was pushed by a **fast\_send** from the frame below (FAST-LOCAL); a FAST-REMOTE callee's frame extends the same path onto another thread. A wait-for edge  $\theta \rightarrow \theta'$  adds the edge from the actor on top of  $\theta$  to an actor already on the path held by  $\theta'$ . Following a wait-for cycle and concatenating these paths yields a cycle in  $G$ .  $\square$

A  $GQ$  run can record the **fast\_send** edges it executes. The recorded graph is a subgraph of  $G$  — it contains only the calls the inputs exercised — so a cycle in it is a real risk to be removed, and the absence of a cycle is evidence only for the paths the backtest covered. A static analysis of the call sites would give all of  $G$ .

Table 3: `send` and `fast_send` on each substrate and between substrates, and the rule of Section 4 each `fast_send` implements.

| Where                   | <code>send</code>                                                    | <code>fast_send</code>                                                                                                           |
|-------------------------|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| CPU, one process        | message into the receiver’s mailbox; a thread per actor or per group | callee’s handler runs on the caller’s thread under the callee’s lock (FAST-LOCAL)                                                |
| C++ ↔ Rust, one process | message converted and enqueued through a C ABI bridge                | callee’s handler runs on the caller’s thread through the bridge (FAST-LOCAL)                                                     |
| FPGA, one process       | —                                                                    | callee is an actor object inside the caller’s process; its handler is inlined into the caller’s pipeline (FAST-LOCAL)            |
| FPGA, between processes | on-chip router into the receiver’s mailbox FIFO                      | request and reply relayed through the host (FAST-REMOTE)                                                                         |
| CPU → FPGA              | envelope over PCIe, router, mailbox FIFO                             | envelope to the actor’s <code>fast_send</code> port, served before its mailbox; the CPU thread waits for the reply (FAST-REMOTE) |
| FPGA → CPU              | envelope over PCIe; a host bridge thread enqueues it                 | request over PCIe; the bridge thread runs the CPU handler inline; the FPGA process stalls until the reply returns (FAST-REMOTE)  |
| GPU (proposed)          | ring buffer in device memory                                         | device function call inside one kernel (FAST-LOCAL); host calls wait on a reply ring (FAST-REMOTE)                               |
| Another machine         | network message                                                      | request/reply RPC; the caller waits (FAST-REMOTE)                                                                                |

**Time.**  $GQ$  reproduces what happens, not when. Latency on each substrate is a separate question (Section 10).

## 7 Mapping `send` and `fast_send` onto substrates

The calculus leaves open what a thread is and how a mailbox is built. Each substrate supplies both, and each supplies a distinct mechanism for each primitive (Table 3).

On an FPGA the two `fast_send` rules correspond to two different hardware arrangements. When the callee is compiled into the caller’s hardware process, its handler becomes part of the caller’s pipeline and runs on the caller’s resource: FAST-LOCAL. When the callee is a separate process, it has its own resource, which the caller waits on: FAST-REMOTE. In the runtime described in Section 8 each actor is its own process, and FAST-LOCAL applies to actor objects that a process contains.

The pattern is the same everywhere. `send` is a queue between two independently scheduled resources. `fast_send` is either an inline call on the caller’s resource, where the substrate lets one resource run another actor’s handler, or a blocking request to a resource dedicated to the callee, where it does not. The calculus of Section 4 covers both with one semantics, and Theorems 6, 7 and 10 hold for any mixture.

A `fast_send` is available between any two substrates where the caller can wait. A CPU thread can wait on a condition. An FPGA process can wait on a blocking stream read, which stalls that

Table 4: Status of each part in Kaspar-HFT. “Semantics” refers to this paper.

| Part                                                                                  | Semantics      | Implementation                                                          | Exercised by                              |
|---------------------------------------------------------------------------------------|----------------|-------------------------------------------------------------------------|-------------------------------------------|
| C++ CPU runtime: <code>send</code> , <code>fast_send</code> , groups                  | §4–5           | implemented                                                             | tests; the Kaspar-HFT trading system [10] |
| Rust runtime; C++ $\leftrightarrow$ Rust <code>send</code> and <code>fast_send</code> | §4–5           | implemented; cross-language <code>fast_send</code> returns no reply yet | tests                                     |
| FPGA runtime and CPU bridge                                                           | §4–5           | implemented as a software model                                         | unit tests in software                    |
| FPGA runtime on hardware                                                              | §4–5           | not yet synthesised                                                     | —                                         |
| GPU runtime                                                                           | Table 3        | design                                                                  | —                                         |
| Global queue and sequencer                                                            | §6             | design; single-runtime instance: actor groups                           | —                                         |
| Per-thread call-chain check                                                           | Proposition 12 | design                                                                  | —                                         |

process alone while the rest of the chip keeps running. What differs between substrate pairs is the cost of the wait.

## 8 Practical implementation in Kaspar-HFT

Kaspar-HFT has actors on three runtimes — C++ on CPU threads, Rust, and an FPGA runtime that currently runs as a software model of the hardware — and a GPU runtime is planned. Each runtime has its own actors, written in its own language; an actor is not recompiled from one runtime to another. What is shared is the model: on every runtime an actor holds references to other actors, whether they run on the same runtime or another, and uses the same two primitives, `send` and `fast_send`, and the same `reply`. The extended actor model of Sections 4 and 5 is the semantics of all of them, and of any mixture of them, with each `fast_send` implemented by FAST-LOCAL or FAST-REMOTE as Table 3 shows. Table 4 separates what is formalised, what is implemented and how each part has been exercised. The rest of this section describes each runtime and then the facility for debugging and simulation.

**CPU runtime.** The Kaspar-HFT C++ runtime [10] implements actors with a mutex-protected turn, a selectable mailbox per actor, `send`, `fast_send` and groups. `fast_send` takes the callee’s lock on the caller’s thread and runs its handler inline; a reply is returned as a value. The delivery flag is private to the framework, but some message fields — the queue depth, the “mailbox empty” flag and the destination — are filled differently for the two deliveries, so a handler that reads them can infer how it was reached. Receiver transparency in Kaspar-HFT is therefore a property of how handlers are written, as in Remark 3, not one the framework enforces. A mailbox may also be configured to busy-poll instead of sleeping, which changes the cost of `send` between threads but not its semantics.

**Rust runtime and interop.** A separate Rust actor runtime hosts Rust actors. C++ and Rust actors reach each other through generated C-ABI bridges, from one message schema; both `send`

and `fast_send` work across the boundary.<sup>4</sup>

**FPGA runtime.** The FPGA runtime is written in Vitis HLS C++. Each actor is a hardware process with a mailbox FIFO, a `fast_send` port served before the mailbox, and a port on which replies to its own remote `fast_sends` arrive; its handlers are selected by a switch on the message identifier. A router delivers messages by destination actor, using a route table fixed when the design is built, and a host link connects the design to PCIe. On the CPU, an `ActorRef` can refer to an FPGA actor; `send` and `fast_send` on it write envelopes to the card, and a bridge thread delivers messages and `fast_send` requests from the card to CPU actors, passing a stand-in for the FPGA actor as the sender so that ordinary replies return to the chip. The design runs today in software, with one thread per hardware process, and its unit tests check `send` through the router, `fast_send` inside a process, `send` and `fast_send` in both directions between CPU and FPGA actors, ordering, error reporting, and the priority of the `fast_send` port over the mailbox.<sup>5</sup>

**GPU runtime.** The same structure maps onto a GPU: actors as warps or thread blocks of a persistent kernel, mailboxes as device-memory rings, `fast_send` inside the kernel as a device function call, and a host bridge over rings in pinned memory. This is a proposed mapping; it has not been built.

**Global queue and sequencer.** Debugging and simulation across runtimes will be provided by a global queue and a *sequencer*, which implement the global-queue semantics *GQ* of Section 6.<sup>6</sup> In this mode every runtime hands each message it sends, whatever its destination, to the sequencer instead of delivering it. The sequencer gives the message a global sequence number and appends it to the global queue. It then delivers messages one at a time: it takes the head of the queue, passes it to the runtime of the receiving actor, and waits until that runtime reports that the turn has ended, including every `fast_send` the turn made, before it delivers the next. Each runtime therefore needs only an adapter with two operations — accept one delivery, report the end of the turn — and keeps its own actors, handlers and `fast_send` mechanism unchanged; an FPGA actor can be driven on the card or in software simulation of its hardware process.

Because deliveries are serialized, a debugger can step through the queue one message at a time and see, before each step, which actor on which runtime will run next and with what message; between steps it can inspect the state of any actor on any runtime, and it can stop on a given actor, message type or runtime. Theorem 16 is what makes this useful, for every call that satisfies Assumption 15: the sequence the debugger shows is one the concurrent system can produce, so a fault found by stepping is real. The same sequence, recorded, replays a backtest exactly, and the `fast_send` calls it executes give the call graph of Proposition 19. What the sequencer cannot show is behaviour such as that of Proposition 18, which needs more than one turn started from a mailbox in progress at once.

---

<sup>4</sup>Kaspar-HFT does not return `fast_send` replies across the language boundary yet; a cross-language `fast_send` delivers the message and returns no value.

<sup>5</sup>The FPGA runtime has not yet been synthesized or run on a card; `fast_send` between two FPGA processes is relayed through the host.

<sup>6</sup>Kaspar-HFT does not implement the cross-runtime global queue and sequencer yet.

## 9 Related work

**Actor semantics.** The operational semantics of Agha, Mason, Smith and Talcott [2] and the four-family semantics of De Koster and De Meuter [6] contain only asynchronous communication. We extend the latter. Our isolation and liveness results are stated against their isolated turn principle.

**Synchronous calls.** ABCL/1 [14] and ABS [9] provide synchronous calls between concurrent objects that block the caller while the callee runs on its own processor, and ABS adds reentrant synchronous calls inside a cog. E [11] restricts immediate calls to one vat. Extended Rebeca [12, 13] is the closest actor language with a formal semantics for synchronous messages, and differs from `fast_send` on each point of Table 1. Its synchronous messages are rendezvous: the receiver must execute an explicit `receive` statement, no handler is invoked, and data flows only from sender to receiver, so there is no reply value. They are confined to one component, while `fast_send` addresses any actor. A rebec blocked in a rendezvous is blocked inside a method, whereas a `fast_send` callee’s turn is an ordinary handler that runs to completion. Its semantics has no execution resources, so the questions answered here for `fast_send` — behaviour when the callee’s resource is shared (Proposition 14) or occupied (Assumption 15), and single-queue simulation (Section 6) — do not arise in it; its aim is compositional model checking. We therefore do not claim that synchronous messages in an actor language are new; the contribution is the particular resource-sensitive primitive and its properties.

One can ask whether `fast_send` could be encoded with rendezvous. An encoding would have the callee wait at a `receive` for the request, run the handler’s code, and hand the result back through a second rendezvous. The callee’s code then runs on the callee’s side, not on the caller’s resource, and the caller’s blocked computation keeps its resource while it waits; when the two share one resource, the argument of Proposition 14 applies unchanged and the encoding deadlocks where FAST-LOCAL completes. The survey of active-object languages by De Boer et al. [5] maps this design space.

**Encodings.** Honda and Tokoro [8] and Boudol [4] encode synchronous in asynchronous communication; Proposition 14 locates where such encodings stop applying once resources are bounded.

**Actors on hardware.** CAL and the StreamBlocks compiler [3] compile dataflow actors to both CPUs and FPGAs. Dataflow actors fire on token availability and have no synchronous call; our FPGA runtime targets the message-passing actor model with both primitives.

## 10 Limitations and future work

- The calculus and proofs are on paper. A mechanisation, for instance by extending the PLT Redex models of [6], would check them.
- The calculus has no failures; a `fast_send` to an actor on a failed resource waits forever. Timeouts and their interaction with isolation are open.
- Detecting cross-thread cycles at run time needs shared state (Proposition 12); its cost on the hot path is not measured.
- Performance is outside the scope of this paper. The cost of `send` and `fast_send` on the C++ runtime is measured in [10]; measuring it across the language boundary, on the FPGA runtime

once it is synthesised and run on a card, and between runtimes, is future work, as is the GPU runtime itself.

- The global queue and sequencer of Section 8 are designs; building them, and a static analysis that computes the call graph of Proposition 19, would turn Theorem 16 and Proposition 19 into tools.

## 11 Conclusion

We defined `fast_send`, a synchronous delivery primitive with explicit execution-resource semantics: the receiver’s turn runs now, on the caller’s resource or on one dedicated to the receiver, under the receiver’s mutual exclusion and without re-entry, and returns a value to a caller that waits for it. Formally, it keeps the actor model’s isolation, makes determinism relative to the values that `fast_send` calls return, and, for terminating handlers under fair scheduling, makes a turn’s completion depend on whether a wait-for cycle is reachable from its thread; asynchronous send alone never creates one. When execution resources are shared, `fast_send` completes where send-then-wait deadlocks, so it is not an abbreviation for it. A single global queue remains a trace-sound simulator with `fast_send`: replay or a debugger that runs every actor, in any language and on any substrate, from one queue shows only behaviour the concurrent system can produce, though not all of it. On the C++ and Rust runtimes and the software model of the FPGA runtime, `send` and `fast_send` are two distinct mechanisms — a queue, and an inline or blocking call — and the same semantic distinction gives direct mappings onto FPGA hardware and GPUs.

## References

- [1] G. Agha. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press, 1986.
- [2] G. A. Agha, I. A. Mason, S. F. Smith, and C. L. Talcott. A foundation for actor computation. *Journal of Functional Programming*, 7(1):1–72, 1997.
- [3] E. Bezati et al. StreamBlocks: A compiler for heterogeneous dataflow computing. Technical report, EPFL, 2021.
- [4] G. Boudol. Asynchrony and the  $\pi$ -calculus. Technical Report 1702, INRIA Sophia-Antipolis, 1992.
- [5] F. de Boer, V. Serbanescu, R. Hähnle, L. Henrio, J. Rochas, C. C. Din, E. B. Johnsen, M. Sirjani, E. Khamespanah, K. Fernandez-Reyes, and A. M. Yang. A survey of active object languages. *ACM Computing Surveys*, 50(5), 2017.
- [6] J. De Koster and W. De Meuter. A formal specification for half a century of actor systems. Manuscript, Vrije Universiteit Brussel, 2023.
- [7] C. Hewitt, P. Bishop, and R. Steiger. A universal modular ACTOR formalism for artificial intelligence. In *IJCAI*, 1973.
- [8] K. Honda and M. Tokoro. An object calculus for asynchronous communication. In *ECOOP ’91*, LNCS 512, pages 133–147, 1991.

- [9] E. B. Johnsen, R. Hähnle, J. Schäfer, R. Schlatte, and M. Steffen. ABS: A core language for abstract behavioral specification. In *Formal Methods for Components and Objects (FMCO 2010)*, LNCS 6957, pages 142–164, 2012.
- [10] V. Maciejewski. Adapting the actor model of concurrency for high-frequency trading: synchronous message delivery (`fast_send`) and a tick-to-book latency study. arXiv:2609.21173, 2026.
- [11] M. S. Miller, E. D. Tribble, and J. Shapiro. Concurrency among strangers: programming in E as plan coordination. In *Trustworthy Global Computing*, 2005.
- [12] M. Sirjani, A. Movaghar, A. Shali, and F. S. de Boer. Extended Rebeca: a component-based actor language with synchronous message passing. In *Application of Concurrency to System Design (ACSD’05)*, pages 212–221, 2005.
- [13] M. Sirjani, F. S. de Boer, and A. Movaghar. Modular verification of a component-based actor language. *Journal of Universal Computer Science*, 11(10):1695–1717, 2005.
- [14] A. Yonezawa, J.-P. Briot, and E. Shibayama. Object-oriented concurrent programming in ABCL/1. In *OOPSLA ’86*, pages 258–268, 1986.